

Contents

NutriChatbot — Full Architecture Document	2
Table of Contents	2
0. Use Cases	3
UC-01 — Patient Walks In for Dietary Counselling	3
UC-02 — Dietitian Uploads New Clinical Guidelines	3
UC-03 — Chronic Disease Management Follow-Up	3
UC-04 — B2B Client Onboarding (New Clinic/Hospital)	4
UC-05 — System Administrator Monitors and Improves Model Quality	4
2. System Flowchart	4
2.1 — Patient Chat Request (End-to-End)	5
2.2 — Patient Login & IC Verification	5
2.3 — Document Ingestion Pipeline	6
2.4 — Embedding Fine-Tuning Pipeline	6
2.5 — Full System Component Map	7
3. High-Level Architecture Diagram	8
2. High-Level Architecture Diagram	9
3. Technology Stack	11
4. Application Entry Point & Startup	11
5. Routing & API Surface	12
Chat API — <code>website_chat_router.py</code>	12
Patient API — <code>app.py</code>	13
Document Management — <code>app.py</code>	13
Admin / Portal	13
6. Authentication & Authorization	13
6.1 B2B API Key Authentication	13
6.2 Patient Authentication	13
7. Patient Identity System	14
7.1 Patient Database Model	14
7.2 Patient Login & IC Verification Flow	15
7.3 Profile Injection into RAG	15
7.4 PatientStore Abstraction	16
7.5 Profile Extractor — Dynamic Data Collection	17
7.6 Nutrition Assessment Priority List — eNCPT v2 Cardiac Schema	19
8. RAG Pipeline — Core Chat Flow	23
8.1 Disease Identification	23
8.2 Conversational Chain Construction	23
8.3 Retriever — Hybrid Multi-Tenant Vector Search	23
8.4 LLM Configuration	24
8.5 Fallback Mechanism	24
8.6 Image Injection	24
9. Embedding Model	24

9.1 Base Model: BAAI/bge-m3	24
9.2 LoRA Adapter Inference	25
10. Database Layer	26
10.1 Relational Database (SQLAlchemy)	26
10.2 Vector Store (pgvector)	26
11. Document Ingestion Pipelines	26
11.1 Base Knowledge Build (build_base_db.py)	26
11.2 Client Document Ingestion (process_client_docs.py)	27
11.3 Document Deletion (document_manager.py)	27
12. Multi-Tenancy Model	27
13. Conversation Memory	28
14. Persona & Prompt Engineering	28
15. Streaming Response Delivery	28
16. UI Layers	29
16.1 Patient-Facing App (patient_app.html)	29
16.2 Admin Dashboard	29
16.3 Client Portal	29
17. Embedding Fine-Tuning Pipeline	30
17.1 Training Data Generation	30
17.2 LoRA Fine-Tuning (finetune_embeddings.py)	30
17.3 Hyperparameter Decisions	31
18. LLM Fine-Tuning Pipeline	34
18.1 Training Data Generation	34
18.2 LoRA Fine-Tuning (Ollama/Unsloth)	34
19. Evaluation Framework	34
20. Testing Strategy	35
Unit Tests	35
Integration Tests	35
21. Deployment	35
Development	35
Production	35
Post-Fine-Tuning Deployment Checklist	35
22. Environment Variables Reference	37
23. Data Flow: End-to-End Request Trace	37
24. Module Reference Table	40

NutriChatbot — Full Architecture Document

Last updated: May 2026

Table of Contents

0. Use Cases

NutriChatbot is designed for the Malaysian outpatient clinical setting where dietitians and clinics serve a multilingual, multicultural patient population with complex chronic disease profiles. The following use cases represent the primary scenarios the system is built to handle.

UC-01 — Patient Walks In for Dietary Counselling

Actor: Patient (Malay/Chinese/Indian, 30–70 years old) **Trigger:** Patient is referred by a doctor or self-presents for nutrition advice **Precondition:** Clinic has registered with NutriChatbot and seeded the patient’s profile (conditions, medications, restrictions)

Flow: 1. Patient opens the app on a clinic tablet or their phone via a QR code link (?key=<clinic_api_key>) 2. Patient types their name → system finds their record 3. Patient confirms identity with their IC number → enters the dashboard 4. Chat opens with a personalised greeting: “Welcome back, Ahmad! I have your profile on file...” 5. Patient asks about breakfast options → system retrieves relevant guidelines from the knowledge base + applies Ahmad’s T2DM + Halal + low-sodium constraints 6. GPT-4-turbo generates a response naming specific local foods (e.g., Oat porridge, Roti wholemeal), avoiding Nasi Lemak and high-sugar Teh Tarik 7. Patient can ask follow-up questions; the system remembers the full conversation context

Value delivered: The dietitian’s clinical notes and the clinic’s own uploaded guidelines shape every answer — not generic internet advice.

UC-02 — Dietitian Uploads New Clinical Guidelines

Actor: Dietitian / Clinic Admin **Trigger:** New Malaysian CPG (Clinical Practice Guideline) or facility-specific diet sheet is published **Precondition:** Dietitian has a B2B API key and access to the Client Portal

Flow: 1. Dietitian logs into /portal/ with their API key 2. Uploads PDF (e.g., *Management of Dyslipidaemia 2023, 6th Edition*) 3. System hashes the file for deduplication, parses and chunks it via Unstructured.io 4. Chunks are embedded via bge-m3 + LoRA adapter and stored in the clinic’s isolated `client_{id}_knowledge` collection 5. All future patient conversations for this clinic now include this document in retrieval

Value delivered: Clinics can keep their knowledge base current without any developer intervention. The knowledge is isolated — other clinics cannot access it.

UC-03 — Chronic Disease Management Follow-Up

Actor: Patient with multiple comorbidities (e.g., T2DM + CKD + Hypertension) **Trigger:** Returning patient with a complex medication-diet interaction question

Example question: “My doctor added Furosemide last week. Are there any foods I need to be more careful about now?”

Flow: 1. Patient logs in → profile loaded (medications list includes Furosemide 40mg OD) 2. RAG retrieves chunks on Furosemide-potassium interaction and fluid restriction guidelines 3. Patient context block in the system prompt already includes Medications: Furosemide 40mg OD and Dietary restrictions: Low potassium, Fluid restriction 1.5L/day 4. GPT-4 cross-references the retrieved guidelines with the patient’s existing profile and generates specific advice — e.g., avoiding high-potassium foods like bananas, coconut water, and herbal soups common in Chinese diet

Value delivered: The system connects the newly retrieved drug-diet interaction knowledge with the patient’s existing profile to give contextually accurate, personalised advice that a generic chatbot could not produce.

UC-04 — B2B Client Onboarding (New Clinic/Hospital)

Actor: Hospital IT Administrator or Clinic Manager **Trigger:** Hospital wants to deploy NutriChatbot for their dietetic department

Flow: 1. Admin contacts NutriChatbot → receives a `nbk_live_{hex}` API key via admin panel 2. Admin configures the URL with `?key=<api_key>` for the clinic tablets/QR codes 3. Admin uploads facility-specific documents (local dietary guidelines, custom diet sheets) via `/portal/` 4. Admin seeds patient records via `seed_patients.py` or the patient management API 5. Clinic goes live — patients can authenticate with IC number and start chatting

Value delivered: Full multi-tenant isolation. Hospital A’s patients and documents are completely separate from Hospital B’s. A single deployment serves multiple clients.

UC-05 — System Administrator Monitors and Improves Model Quality

Actor: ML Engineer / System Admin **Trigger:** Periodic quality review, or after ingesting a new batch of documents

Flow: 1. Run `python eval_ragas.py --out results.json` to score the current pipeline on 25 ground-truth questions 2. Review faithfulness (hallucination rate), context_precision (retrieval quality), and answer_relevancy scores 3. If context scores are low: generate more training data (`generate_embedding_training_data.py`) and re-run embedding fine-tuning (`finetune_embeddings.py`) 4. If answer quality is low: update the system prompt in `chain_factory.py` or switch to a stronger LLM 5. Re-run eval to confirm improvement

Value delivered: A closed feedback loop — the system can be objectively measured and iteratively improved without subjective testing.

2. System Flowchart

The following Mermaid diagrams show how all components connect across the four main journeys: patient chat, document ingestion, patient login, and the embedding pipeline.

2.1 — Patient Chat Request (End-to-End)

flowchart TD

```
A[Patient sends question\nvia patient_app.html]) --> B[POST /chat/  
get_response\nX-API-Key header]  
B --> C{API Key valid?}  
C -- No --> D[HTTP 401 Unauthorised]  
C -- Yes --> E[Resolve patient profile\nfrom patient_id]  
E --> F{Profile found\nin DB?}  
F -- No --> G[identify_target_disease\nLLM call to extract condition]  
F -- Yes --> H[Build patient_context string\nname, age, BMI, conditions,  
\nmeds, restrictions, notes]  
G --> I[create_conversational_chain\nchain_factory.py]  
H --> I  
I --> J[MergedRetriever\nvector_store.py]  
J --> K[Embed query\nbge-m3 + LoRA adapter\nembeddings.py]  
K --> L[Cosine search\nbase_knowledge\npgvector]  
K --> M[Cosine search\nclient_id_knowledge\npgvector]  
L --> N[Merge + deduplicate\ntop ~8 chunks]  
M --> N  
N --> O[Build prompt\nSystem: ADIME + patient block\nHistory: prior  
turns\nContext: retrieved chunks\nHuman: question]  
O --> P{USE_OLLAMA?}  
P -- false --> Q[GPT-4-turbo\nOpenAI API]  
P -- true --> R[Local Ollama model\nnuTriBot / adime-final]  
Q --> S[Parse response\nfor IMAGE markers\nimage_handler.py]  
R --> S  
S --> T[StreamingResponse\nSSE char-by-char]  
T --> U[Patient sees answer\nin real time]  
U --> V[Append to session memory\nsession_store]
```

2.2 — Patient Login & IC Verification

flowchart TD

```
A[Patient opens app\npatient_app.html]) --> B{API key in URL\n?key=...}  
B -- Yes --> C[Show Connected badge\nSkip key input]  
B -- No --> D[Show key input field]  
C --> E[Patient types name]  
D --> E  
E --> F[POST /patient/login\nname lookup]  
F --> G{Match found?}  
G -- No match --> H[Show error\nCheck spelling or\ncontact clinic]  
G -- Multiple matches --> I[Show IC input\nfor disambiguation]  
G -- Single match --> J[Patient enters IC number\nauto-formatted YYMMDD-SS-XXXX]  
J --> K[POST /patient/login\nwith ic_number]  
K --> L{IC matches\nrecord?}  
L -- No --> M[Show error\nIC not matched]  
L -- Yes --> N[Load full patient profile\nfrom DB]  
N --> O[Transition to Dashboard]
```

```

0 --> P[Show patient sidebar\nconditions, meds,\ndiet, allergies]
0 --> Q[Open chat with\npersonalised greeting]

```

2.3 – Document Ingestion Pipeline

flowchart TD

```

    A([Admin uploads PDF/DOCX\nvia /portal/ or API]) --> B[SHA-256 hash\nfor deduplication]
    B --> C{Hash already\nin DB?}
    C -- Yes --> D([Skip – return status: skipped])
    C -- No --> E[Unstructured.io partition\nstrategy: fast]
    E --> F{Text\nextracted?}
    F -- No --> G[Retry with hi_res\nTesseract OCR\nlanguages: eng + msa]
    F -- Yes --> H[chunk_by_title\nmax 1500 chars\nmin 500 chars]
    H --> I[Strip NUL bytes\nfrom each chunk]
    I --> J[Embed all chunks\nbge-m3 + LoRA adapter\nCPU]
    J --> K[Batch insert into pgvector\nclient_id_knowledge collection\nbatch size 100]
    K --> L[Record metadata in PostgreSQL\ndocument_metadata table]
    L --> M([Return chunk_count\nand status: completed])

```

```

    subgraph Base Knowledge Build
    N([Admin runs build_base_db.py\nBASE_DOCS_DIR=...]) --> O[Scan directory\nskip ._ files]
    O --> P{File in\nfile_tracker.json?}
    P -- Up to date --> Q([Skip])
    P -- New or changed --> R[ProcessPoolExecutor\n4 parallel workers]
    R --> S[Same partition +\nchunk pipeline as above]
    S --> T[Embed + insert into\nbase_knowledge collection\nbatch size 500]
    T --> U[Update file_tracker.json]
    end

```

2.4 – Embedding Fine-Tuning Pipeline

flowchart TD

```

    A([pgvector base_knowledge\n35,000+ chunks]) --> B[generate_embedding_training_data.py\nfetch all chunks via psycopg2]
    B --> C[For each chunk:\ncall Ollama / GPT-4\ngenerate 2 questions]
    C --> D[Save as JSONL pairs\nanchor: question\npositive: chunk text]
    D --> E[~/data/embedding_train.jsonl\n~62,000 pairs]
    D --> F[~/data/embedding_val.jsonl\n~7,000 pairs]

    E --> G[finetune_embeddings.py]
    F --> G
    G --> H[Load BAAI/bge-m3\nbase model]
    H --> I[Attach LoRA adapter\nr=16, alpha=32\ntargets: query + value]
    I --> J[MultipleNegativesRankingLoss\nin-batch negatives\neffective batch 32]
    J --> K[Train 3 epochs\nbf16, RTX 3050\n~45-60 min]
    K --> L[Save adapter weights\n~/models/embedding_lora/]

```

```

L --> M[Set EMBEDDING_ADAPTER_PATH\nin .env]
M --> N[embeddings.py loads\nbase model + adapter on CPU]
N --> O[Rebuild vector store\nbuild_base_db.py]
O --> P([All future queries use\nfine-tuned embeddings])

```

2.5 — Full System Component Map

flowchart LR

subgraph Clients

```

    PA([Patient App\npatient_app.html])
    CP([Client Portal\n/portal/])
    AD([Admin Panel\n/admin/])
    API([B2B API\nexternal integrations])

```

end

subgraph FastAPI [FastAPI – app.py :8000]

```

    WC[website_chat_router\n/chat/get_response]
    PR[patient endpoints\n/patient/login\n/patients/]
    DM[document endpoints\n/upload_documents/\n/documents/]
    CR[client_portal_router]
    AR[admin_router]

```

end

subgraph Auth

```

    DEP[dependencies.py\nget_api_client\nget_db]

```

end

subgraph RAG [RAG Pipeline]

```

    RAG1[rag.py\norchestration]
    RAG2[chain_factory.py\nLCEL chain + prompt]
    RAG3[vector_store.py\nMergedRetriever]
    RAG4[llm.py\nChatOpenAI / ChatOllama]

```

end

subgraph Embeddings

```

    EM[embeddings.py\nsingleton]
    BGE[BAAI/bge-m3\nbase model]
    LORA[LoRA adapter\n~/models/embedding_lora/]

```

end

subgraph Storage

```

    PG[(PostgreSQL\napi_clients\npatients\ndocument_metadata)]
    VEC[(pgvector\nbase_knowledge\nclient_id_knowledge)]

```

end

subgraph LLM

```

    GPT[GPT-4-turbo\nOpenAI API]
    OLL[Ollama\nnuTriBot / adime-final]

```

end

```

PA --> WC
PA --> PR
CP --> CR
AD --> AR
API --> WC
API --> DM

WC --> DEP
PR --> DEP
DM --> DEP
CR --> DEP
AR --> DEP

DEP --> PG
WC --> RAG1
RAG1 --> RAG2
RAG1 --> RAG3
RAG2 --> RAG4
RAG3 --> EM
EM --> BGE
EM --> LORA
RAG3 --> VEC
RAG4 --> GPT
RAG4 --> OLL
PR --> PG
DM --> PG
DM --> VEC

```

3. High-Level Architecture Diagram

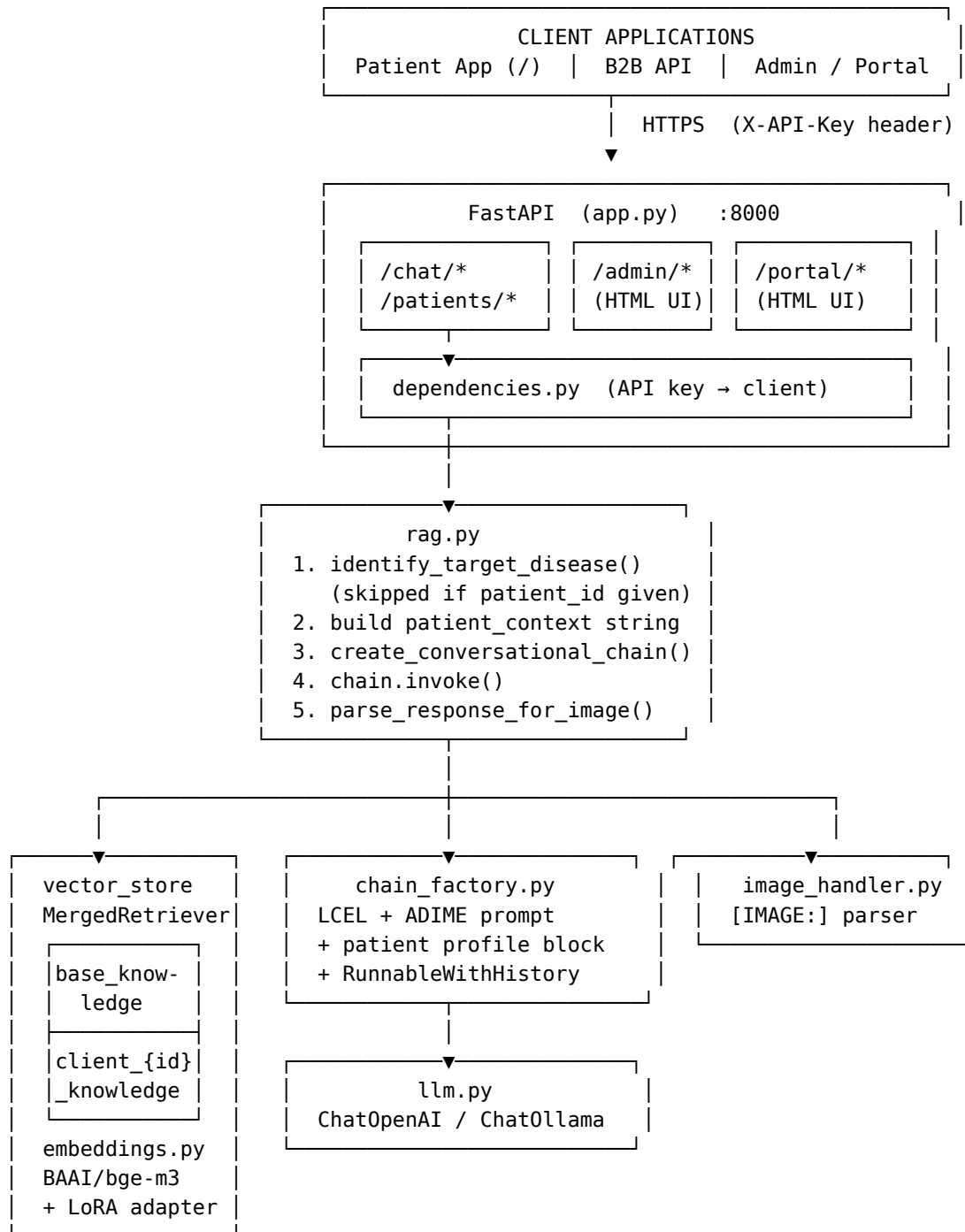
NutriChatbot is a multi-tenant B2B SaaS nutrition counselling chatbot built on a Retrieval-Augmented Generation (RAG) pipeline. Its core role is to act as an AI dietitian, guiding patients through the **ADIME nutrition care process** (Assessment → Diagnosis → Intervention → Monitoring & Evaluation) with specific awareness of the Malaysian multicultural food context.

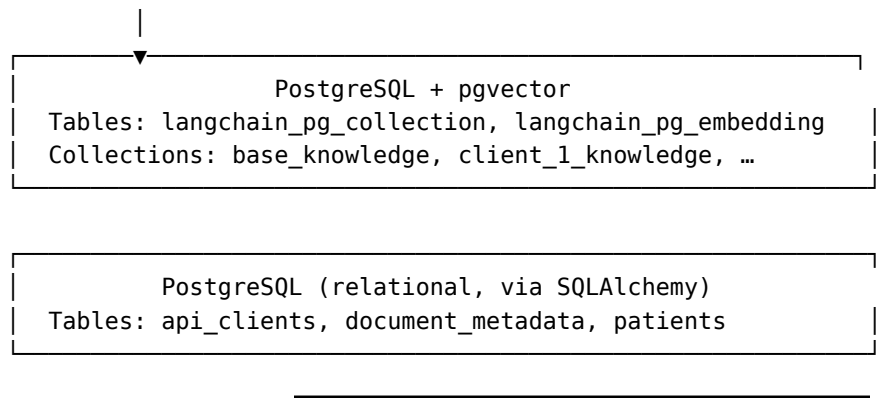
Key design properties:

- **Multi-tenant:** Each B2B client gets an isolated knowledge base alongside a shared foundational knowledge base. A single API key identifies the tenant for every request.
- **Patient-aware:** Patients can log in with their name and IC number. Their full medical profile (conditions, medications, dietary restrictions, allergies, BMI) is automatically injected into every LLM prompt for personalised counselling.
- **Hybrid RAG:** Every query simultaneously retrieves from a shared `base_knowledge` collection (medical/nutrition guidelines) and the client's own `client_{id}_knowledge` collection, then merges and deduplicates.
- **Multilingual:** Embedding model (BAAI/bge-m3) natively handles both English and Bahasa Melayu documents and queries. OCR pipeline includes Malay language support.

- **Dual LLM backend:** Defaults to OpenAI gpt-4-turbo but can switch to a locally hosted LoRA fine-tuned model (via Ollama) by setting one environment variable.
- **Streaming:** Responses are streamed as Server-Sent Events (SSE) for real-time user experience.

2. High-Level Architecture Diagram





3. Technology Stack

Layer	Technology	Version / Notes
Web framework	FastAPI	Latest via pip
ASGI server	Uvicorn + Gunicorn	<code>uvicorn.workers.UvicornWorker</code> in production
Language model (cloud)	OpenAI gpt-4-turbo	Configurable via <code>OPENAI_MODEL</code>
Language model (local)	Ollama + custom LoRA model	Toggled via <code>USE_OLLAMA=true</code>
Embedding model	BAAI/bge-m3	570M params, 1024-dim, multilingual, runs on CPU
Embedding fine-tuning	PEFT + sentence-transformers 3.x	LoRA on bge-m3, RTX 3050 4GB
LLM orchestration	LangChain 0.1.x LCEL	<code>langchain==0.1.20</code> , <code>langchain-core==0.1.52</code>
Vector store	pgvector on PostgreSQL	<code>langchain-community PGVector</code>
Relational DB	PostgreSQL	SQLAlchemy ORM
Document parsing	Unstructured.io ≥ 0.11	Supports PDF, DOCX, OCR fallback
OCR (Malay)	Tesseract + <code>tesseract-ocr-msa</code>	For scanned Bahasa Melayu PDFs
Password hashing	werkzeug PBKDF2	<code>generate_password_hash</code> / <code>check_password_hash</code>
Session caching	Redis (optional)	In-memory dict fallback
Evaluation	RAGAS	<code>faithfulness</code> , <code>answer_relevancy</code> , <code>context_precision</code> , <code>context_recall</code>
Container	Docker	<code>python:3.11-slim</code> base

4. Application Entry Point & Startup

File: `app.py`

On startup:

1. Loads environment variables from `.env` via `python-dotenv`.

2. Creates a FastAPI instance.
3. Mounts /images as a static file directory pointing to data/images/.
4. Registers a startup_event that calls db.create_db_and_tables() to ensure all relational tables exist (including the patients table).
5. Adds CORS middleware with allow_origins=["*"].
6. Registers routers: /admin, /portal, /chat.
7. Defines document management REST endpoints directly on the app.
8. Serves patient_app.html at GET / and the legacy test UI at GET /dev.

Route change (April 2026): The root / now serves the production patient-facing app. The original developer test UI was moved to /dev. This was done because the patient app is the primary end-user interface and should occupy the canonical URL.

5. Routing & API Surface

Chat API — website_chat_router.py

Method	Path	Auth	Description
POST	/chat/get_response	X-API-Key	Streaming SSE chat response

Request body (ChatRequest):

```
{
  "question": "What should I eat for breakfast?",
  "session_id": "user-abc-session-1",
  "profile": { ... },
  "patient_id": 3
}
```

patient_id (added April 2026): When provided, the server auto-loads the patient's full medical record from the database and constructs the profile dict automatically. This removes the burden from the caller to manage profile data and ensures the profile is always authoritative and up-to-date from the DB.

profile (legacy): A manually-supplied profile dict. Still supported. When both patient_id and profile are provided, patient_id takes precedence.

Patient API — `app.py`

Method	Path	Auth	Description
POST	<code>/patient-login</code>	X-API-Key	Name-based patient lookup (step 1 of login)
POST	<code>/patient-verify</code>	X-API-Key	IC number verification (step 2 of login)
GET	<code>/patients/</code>	X-API-Key	List all patients for this client
GET	<code>/patients/{id}</code>	X-API-Key	Get a specific patient's full record

Document Management — `app.py`

Method	Path	Auth	Description
POST	<code>/upload_documents/</code>	X-API-Key	Upload one or more PDF/DOCX files
GET	<code>/documents/</code>	X-API-Key	List all documents for this client
DELETE	<code>/documents/{id}</code>	X-API-Key	Delete a document from DB and vector store

Admin / Portal

Same as previous — see sections 16.2 and 16.3.

6. Authentication & Authorization

6.1 B2B API Key Authentication

All client-facing endpoints use `get_api_client()` as a FastAPI Depends injection (file: `dependencies.py`).

Key format: `nbk_live_{64 hex chars}` — generated by `secrets.token_hex(32)` prefixed with `nbk_live_`.

Storage: Only the werkzeug PBKDF2 hash is stored. The raw key is shown once at creation. Hash comparison is sequential (not indexed) to prevent timing-based enumeration.

6.2 Patient Authentication

Patients authenticate via a two-step UI flow (not a JWT/session token system — it is a stateless lookup on each chat request):

1. **Step 1 — Name lookup:** POST /patient-login with { name, ic_number? }. Returns the matched patient record or a multi-match indicator.
2. **Step 2 — IC verification:** POST /patient-verify with { patient_id, ic_number }. Confirms identity before granting access to the chat dashboard.

Why IC number, not password? Malaysian healthcare workflows use IC (Identity Card) number as the primary patient identifier. This removes the need for patients to manage a separate password, reducing friction for elderly or less tech-savvy users.

7. Patient Identity System

7.1 Patient Database Model

File: database.py — Patient SQLAlchemy model

```
patients table
├─ id                Integer PK
├─ client_id         Integer FK → api_clients.id (CASCADE DELETE)
├─ name              String (NOT NULL)
├─ ic_number          String (indexed) – Malaysian IC format: YYMMDD-SS-XXXX
├─ age                Integer
├─ gender             String – "Male" / "Female"
├─ ethnicity          String – "Malay" / "Chinese" / "Indian"
├─ weight_kg          Float
├─ height_cm          Float
├─ conditions          JSON – list of condition strings
├─ medications         JSON – list of medication strings
├─ dietary_restrictions JSON – list of restriction strings
├─ allergies           JSON – list of allergen strings
├─ notes              String – free-text clinical notes
├─ username           String (UNIQUE, indexed)
└─ hashed_password    String – werkzeug PBKDF2
```

Key design decisions:

- conditions, medications, dietary_restrictions, allergies are stored as **JSON arrays** (not normalised relational tables). This avoids schema complexity for a field whose values vary widely per patient and change infrequently. The trade-off is no indexed querying on individual conditions, which is acceptable since the primary query pattern is “load all conditions for one patient”, not “find all patients with condition X”.
- ic_number is stored as a plain string but normalised on lookup (get_patient_by_ic strips dashes and spaces before comparing) to handle user input formatting inconsistencies.
- client_id FK with CASCADE DELETE ensures that deleting a B2B client automatically removes all their patient records, maintaining referential integrity without manual cleanup.

CRUD functions added to database.py:

Function	Description
<code>add_patient(db, client_id, ...)</code>	Hashes password, inserts Patient row
<code>get_patient(db, patient_id)</code>	Lookup by PK
<code>get_patient_by_username(db, username)</code>	Exact match
<code>get_patient_by_ic(db, ic_number)</code>	Normalises IC, exact match
<code>get_patients_by_client(db, client_id)</code>	All patients for a tenant
<code>check_patient_login(db, username, password)</code>	Hash-verified login
<code>patient_to_profile_dict(patient)</code>	Converts ORM object to RAG profile dict

The `patient_to_profile_dict` function uses the key "condition" (not "conditions") to match the pre-existing schema expected by `rag.py`'s profile handling logic.

7.2 Patient Login & IC Verification Flow

File: `patient_app.html` (frontend) + `app.py` (backend)

Screen 1 (Login)

- └ User enters name
- └ POST `/patient-login` → DB lookup by name (exact → ILIKE fallback)
 - └ Single match → proceed to Screen 2
 - └ Multiple matches → prompt for IC to narrow down
- └ No match → error

Screen 2 (IC Verification)

- └ User enters IC number (auto-formatted as YYMMDD-SS-XXXX)
- └ POST `/patient-verify` → DB lookup by IC, confirms `patient_id` match
 - └ Match → proceed to Screen 3
 - └ No match → error

Screen 3 (Dashboard + Chat)

- └ Patient profile shown in sidebar (conditions, meds, diet, allergies)
- └ All chat requests include `patient_id` in body
- └ Server auto-loads profile from DB on each request

Why two-step? A single name lookup would fail for common Malaysian names (e.g., "Ahmad" appears frequently). IC number as the second factor provides unambiguous identity verification without requiring a traditional password.

7.3 Profile Injection into RAG

File: `rag.py`

When a patient is identified, the profile dict is converted to a structured `patient_context` string injected into the LLM system prompt:

Name: Ahmad Fadzillah bin Roslan
 Age: 52
 Gender: Male
 Ethnicity: Malay
 Weight: 88.0kg, Height: 168.0cm, BMI: 31.2
 Conditions: Type 2 Diabetes, Hypertension
 Medications: Metformin 500mg BD, Lisinopril 10mg OD, Aspirin 100mg OD
 Dietary restrictions: Halal only, Low sodium, Low simple carbohydrates
 Clinical notes: Office administrator, sedentary lifestyle. Eats Nasi Lemak...

This block is inserted into the chain_factory.py system prompt between the role definition and the ADIME framework instructions. The LLM is instructed to address the patient by name, tailor food suggestions to their ethnicity, and respect all restrictions without exception.

Why inject into system prompt vs. human message? System prompt injection means the profile context persists across the entire conversation and cannot be overridden by user messages. Injecting into the human message would require repeating it every turn and risks the model treating it as user input rather than clinical context.

7.4 PatientStore Abstraction

File: patient_store.py

The PatientStore ABC decouples the bot from any specific patient data backend. All code that reads or writes patient profiles goes through this interface.

```

class PatientStore(ABC):
    def get_profile(self, patient_id: int) -> dict | None: ...
    def update_supplementary_fields(
        self, patient_id: int, updates: dict, source_session_id: str | None =
None
    ) -> dict: ...
  
```

Implementation	File	Status
LocalPatientStore	local_patient_store.py	Current (dev/staging) — wraps SQLAlchemy ORM
RemotePatientStore	(not yet built)	Future — will call hospital REST/FHIR API

SUPPLEMENTARY_FIELDS whitelist (defined in patient_store.py): an explicit set of every DB column the extractor is allowed to write. Any key not in this set is silently rejected by update_supplementary_fields. This prevents the extractor from ever touching clinical fields (conditions, medications, allergies, etc.) that are owned by the hospital system.

Current whitelist: fluid_intake_ml, alcohol_per_week, supplements, religion, tobacco_status, meals_per_day, snacks_per_day, processed_food_freq, fast_food_freq, self_prepared_freq, caffeine_mg_per_day, sugar_drinks_ml, activity_freq, activity_minutes, activity_intensity, food_avoidance, nutrition_knowledge,

readiness_to_change, sodium_awareness, fat_intake_level, fat_sources,
medication_compliance, activity_types.

Provenance: Every write through `update_supplementary_fields` records metadata in the `extractor_metadata` JSON column: `{field: {confidence, last_updated, source_session_id}}`. This creates an audit trail of what the bot inferred and from which session.

7.5 Profile Extractor — Dynamic Data Collection

File: `extractor.py`

The extractor passively analyses every incoming patient message and extracts new supplementary profile fields from what the patient says. It is:

- **Conservative** — only extracts what is explicitly stated, never infers or assumes
- **Additive** — only fills fields that are currently empty (`None`, `[]`, or `""`), never overwrites
- **Whitelist-constrained** — only writes to `SUPPLEMENTARY_FIELDS`; rejected at the `PatientStore` layer regardless

Pipeline:

```

Patient message
|
▼
_build_field_descriptions()    ← format EXTRACTOR_FIELDS for prompt
_build_known_summary(profile) ← show LLM what is already filled
|
▼
EXTRACTION_PROMPT.format(...) ← construct structured extraction prompt
|
▼
call_ollama_extractor(prompt)  ← qwen2.5:32b, temperature=0.0, num_predict=200
|
▼
_strip_json_response(raw)     ← strip markdown fences / LLM noise
|
▼
json.loads(cleaned)           ← parse JSON dict; return {} on decode error
|
▼
_validate_extraction(dict)    ← type checks, range checks, allowed_values
enforcement
|
▼
_filter_already_filled(dict)   ← drop any field that already has a value
|
▼
return dict of new fields → PatientStore.update_supplementary_fields()
```

Key functions:

Function	Description
<code>extract_from_message(message, current_profile)</code>	Main entry point. Returns {} on any error — never raises.
<code>call_ollama_extractor(prompt)</code>	POST to Ollama /api/generate; temperature=0.0; num_predict=200
<code>_build_field_descriptions()</code>	Formats EXTRACTOR_FIELDS list for inclusion in the prompt
<code>_build_known_summary(profile)</code>	Summarises already-filled fields so the LLM doesn't re-extract them
<code>_strip_json_response(text)</code>	Removes ```json ``` fences and whitespace from LLM output
<code>_validate_extraction(extracted)</code>	Type-checks and range-validates each extracted value; drops invalid entries
<code>_filter_already_filled(extracted, profile)</code>	Drops fields that already have a non-null, non-empty value

EXTRACTOR_FIELDS (current — v1, 5 active fields):

Field	eNCPT Code	Type	Validation
fluid_intake_ml	FH-1.2.1.1.1	int (mL/day)	0 < value < 10000
alcohol_per_week	FH-1.4.1.1	int (drinks/week)	0 ≤ value < 200
supplements	FH-3.2.1	list of strings	all elements non-empty str
religion	CH-3.1.7	string	non-empty, len < 100
tobacco_status	CH-1.1.10	string	one of: Never smoked, Current smoker, Former smoker

Pending (v2 cardiac — fields exist in DB, extractor not yet extended): fat_intake_level (FH-1.5.1.1), fat_sources (FH-1.5.1.2), medication_compliance (FH-3.1.1.1), activity_types (FH-7.3.1.1), sodium_awareness (FH-1.5.6.1).

Prompt design decisions: - temperature=0.0 — deterministic; medical data extraction must not vary between calls. - num_predict=200 — capped output forces a small JSON object, not paragraphs. - Already-known fields are listed in the prompt so the LLM skips them rather than re-extracting the same value. - The prompt explicitly forbids prose: *“Return ONLY a JSON object. No prose. No markdown. Just JSON.”*

Why qwen2.5:32b for extraction (not CLaRa-7B)? CLaRa is optimised for compressed-context retrieval and answer generation. qwen2.5:32b handles structured JSON output tasks far more reliably due to its larger capacity and instruction-following training. The extractor call is short (200 tokens max) and runs once per message, so the extra latency is acceptable.

Integration point: `website_chat_router.py` calls `extract_from_message()` after the RAG response is generated (non-blocking — extraction happens in the background for non-streaming path; currently synchronous for streaming path). If new fields are found, `patient_store.update_supplementary_fields()` writes them with session provenance.

7.6 Nutrition Assessment Priority List — eNCPT v2 Cardiac Schema

Files: `data/encpt/encpt_curated.json` (machine-readable), `data/encpt/encpt_curated.md` (dietitian review doc) **Version:** v2.0-cardiac | **Total fields:** 34 | **Languages:** EN + Bahasa Malaysia

The eNCPT (electronic Nutrition Care Process Terminology) schema defines what supplementary patient data the chatbot collects during conversation. Every field maps to an eNCPT 2020 code for EHR integration. The v2 schema was specialised for cardiac patients per dietitian consultation in May 2026.

Schema generation workflow:

```
# Edit the schema definition:
vim data/encpt/build_curated_schema.py

# Regenerate the JSON and human-readable markdown:
python data/encpt/build_curated_schema.py
python data/encpt/json_to_md.py data/encpt/encpt_curated.json data/encpt/
encpt_curated.md

# Send encpt_curated.md to the supervising dietitian for review.
```

Priority tiers:

Tier	Description	Field count	Bot behaviour
Tier 1 — Critical	Must collect for safe cardiac dietary advice	7	Do not give nutritional recommendations without these
Tier 2 — Important	Significantly improves personalisation	17	Collect during the first few sessions
Tier 3 — Nice to Have	Adds context for long-term care	10	Collect opportunistically during conversation

Tier 1 — Critical (7 fields):

eNCPT Code	Field	DB Column	Relevant For
FH-1.2.1.1.1	Total fluid intake (mL/day)	fluid_intake_ml	Heart Failure, CKD, Hypertension
FH-1.4.1.1	Alcohol intake (drinks/week)	alcohol_per_week	All patients
CH-1.1.10	Tobacco use	tobacco_status	Cardiac, HTN, Dyslipidaemia
FH-1.5.1.1 ☒	Total fat intake (qualitative: low/moderate/high)	fat_intake_level	Cardiac, Dyslipidaemia
FH-1.5.6.1 ↑	Sodium intake / awareness	sodium_awareness	Cardiac, HTN, Heart Failure, CKD
FH-1.6	Food allergies and intolerances	extractor_food_allergies (pending migration)	All patients
CH-3.1.7	Religion (affects diet)	religion	All patients

☒ New in v2. ↑ Promoted from Tier 2 in v2 (critical for cardiac).

Tier 2 — Important (17 fields):

eNCPT Code	Field	DB Column	Relevant For
FH-3.1.1.1 ☒	Medication compliance (good/variable/poor)	medication_compliance	Cardiac, HTN, Heart Failure
FH-1.5.1.2 ☒	Fat type sources (raw food sources)	fat_sources	Cardiac, Dyslipidaemia
FH-7.3.1	Physical activity frequency	activity_freq	All
FH-7.3.2	Physical activity duration (minutes)	activity_minutes	All
FH-7.3.1.1 ☒	Type of physical activity	activity_types	Cardiac, Heart Failure
FH-7.3.3	Physical activity intensity (light/moderate/vigorous)	activity_intensity	Cardiac, Heart Failure
FH-1.4.3.1	Caffeine intake (mg/day)	caffeine_mg_per_day	Cardiac, HTN, Atrial Fibrillation
FH-1.2.2.3.1.1	Meals per day	meals_per_day	Diabetes, HTN
FH-1.2.2.3.1.2	Snacks per day	snacks_per_day	Diabetes, Obesity
FH-1.2.2.2.5	Processed food intake frequency	processed_food_freq	Cardiac, HTN, Dyslipidaemia, Obesity
FH-1.2.2.2.6	Fast food / takeaway frequency	fast_food_freq	Cardiac, HTN, Dyslipidaemia, Obesity, Diabetes
FH-1.2.2.2.7	Self-prepared food frequency	self_prepared_freq	All
FH-1.2.1.1.1.3	Sugar-sweetened beverage intake (mL/day)	sugar_drinks_ml	Diabetes, Obesity, Cardiac
FH-5.2.1	Food avoidance (voluntary)	food_avoidance	All
FH-5.4.1	Cultural/religious eating practices	(derived from religion)	All
FH-4.1.3	Nutrition knowledge (1–5 scale)	nutrition_knowledge	All
FH-4.2.8	Readiness to change	readiness_to_change	All

Tier 3 – Nice to Have (10 fields):

eNCPT Code	Field	Relevant For	Note
FH-3.2.1 ↓	Vitamin/mineral supplement intake	All	supplements column; demoted from T1 for cardiac
FH-6.2.1	Food availability	All	
FH-1.4.1.4	Alcohol pattern on drinking days	Liver, HTN, Cardiac	
FH-6.2.3	Access to food prep equipment	All	
FH-5.4.2	Eating environment (alone/family/etc.)	All	
CH-3.1.9	Daily stress level (1–10)	Cardiac, HTN	
CH-3.1.6	Occupation	All	
CH-3.1.4	Social and medical support	All	
FH-1.5.4.5	Fibre estimated intake	Diabetes, Dyslipidaemia, Cardiac	
FH-8.1	Nutrition quality of life (1–5)	All	

↓ Demoted from Tier 1 in v2 (less critical for cardiac).

v2 changes from v1 (per dietitian, May 2026): - **New fields:** fat_intake_level (→ T1), medication_compliance (→ T2), fat_sources (→ T2), activity_types (→ T2) - **Promoted:** sodium_awareness T2 → T1 (direct driver of hypertension and fluid retention) - **Demoted:** supplements T1 → T3 (lower priority for cardiac focus specifically)

Personalization levels (from dietitian, May 2026) – stored in personalization_level column:

Level	Patient Profile	Content Scope
L0	No risk, no history, no functional limits	Full spectrum incl. vigorous activity and performance goals
L1	Emerging/moderate risk (early HTN, elevated BMI), no functional limits	Structured, safety-aware; moderation and do/don't boundaries
L2	Established conditions, physical limitations, higher CV risk	Low-intensity, symptom monitoring, strict stop conditions
L3	High clinical risk, recent cardiac events, disability	Medical oversight only; emergency education; minimal activity guidance

Mock patient assignments: P1 (Ahmad) = L2, P2 (Lim) = L2, P3 (Kavitha) = L1, P4 (Hafizuddin) = L1, P5 (Tan) = L2, P6 (Nurul) = L0, P7 (Rajendran) = L3.

8. RAG Pipeline — Core Chat Flow

File: rag.py

`get_rag_response(question, client_id, chat_session_id, profile=None)` orchestrates four steps:

8.1 Disease Identification

If no profile is passed, `identify_target_disease(question)` sends a zero-shot LLM prompt extracting the primary health condition from the user's message (e.g., "hypertension"). If a profile is passed, this step is skipped and the disease context is built directly from the profile's condition list.

8.2 Conversational Chain Construction

File: chain_factory.py

`create_conversational_chain(client_id, target_disease, patient_context="")` builds a stateful LangChain LCEL chain:

```
RunnablePassthrough.assign(context = retriever)
| ChatPromptTemplate([system_with_patient_block, chat_history, human])
| LLM
| StrOutputParser()
```

Wrapped in `RunnableWithMessageHistory` for per-session memory.

8.3 Retriever — Hybrid Multi-Tenant Vector Search

File: vector_store.py

MergedRetriever calls both base_knowledge and client_{id}_knowledge pgvector collections (top-5 each), merges, and deduplicates by page_content. Cosine similarity search using normalized embeddings.

8.4 LLM Configuration

File: llm.py

- **Default:** ChatOpenAI(model="gpt-4-turbo", temperature=0.5, max_tokens=1500)
- **Local:** ChatOllama(model=OLLAMA_MODEL, temperature=0.5, num_predict=1500)

temperature=0.5: Balances factual precision (important for medical advice) with natural conversational variation. Pure 0.0 produces robotic repetitive responses; 0.7+ risks hallucination on clinical details.

max_tokens=1500: Sufficient for 2–4 paragraph dietitian responses. Capped to prevent runaway costs and avoid context-window exhaustion during long sessions.

8.5 Fallback Mechanism

If the RAG answer contains "i don't know", "i am not sure", or "i cannot answer", or is empty, the question is re-issued directly to the LLM without retrieval context. This handles questions that fall outside the knowledge base without returning an unhelpful non-answer.

8.6 Image Injection

File: image_handler.py

Post-processes LLM output for [IMAGE: <query>] markers, matches against data/image_annotations.csv by keyword intersection, and returns the best-matching image filename.

9. Embedding Model

9.1 Base Model: BAAI/bge-m3

File: embeddings.py

```
HuggingFaceEmbeddings(  
    model_name="BAAI/bge-m3",  
    model_kwargs={"device": "cpu"},  
    encode_kwargs={"normalize_embeddings": True},  
)
```

Model specifications:

Property	Value
Parameters	570M
Output dimension	1024
Context window	8,192 tokens
Languages	100+ (including Bahasa Melayu)
Architecture	XLNet encoder
MTEB score	54.9 (multilingual average)

Why bge-m3 over the previous bge-small-en-v1.5?

The original bge-small-en-v1.5 (33M params, 384-dim, English-only) was chosen for its small footprint. It became inadequate when:

1. A second document folder containing Bahasa Melayu PDFs was ingested. English-only models tokenise Malay text poorly and produce degraded embeddings.
2. Malaysian patient names, local food terms (Nasi Lemak, Rendang, Teh Tarik), and clinical notes contain extensive code-switching between English and Malay that the small model failed to encode meaningfully.

bge-m3 was selected over other multilingual alternatives (e.g., multilingual-e5-large, paraphrase-multilingual-mpnet) because: - It is the highest-scoring publicly available multilingual model on MTEB as of early 2026. - It supports 8,192-token context (vs. 512 for most competitors), crucial for long clinical notes and multi-page medical guidelines. - It is published by BAAI (Beijing Academy of AI) with a permissive license. - Trade-off accepted: 2.3GB disk / ~1.5GB RAM vs. 130MB for bge-small. This is acceptable because the model runs on CPU as a singleton loaded once at startup.

normalize_embeddings=True: L2-normalises all vectors to unit length. This converts cosine similarity search into a simple dot product, which pgvector computes faster. All embeddings in the vector store were re-generated after this model change.

9.2 LoRA Adapter Inference

When EMBEDDING_ADAPTER_PATH is set and the directory exists, get_embedding_function() calls _load_lora_embedding() instead of the base HuggingFaceEmbeddings:

```
base_hf_model = AutoModel.from_pretrained(base_model_name)
peft_model = PeftModel.from_pretrained(base_hf_model, adapter_dir)
```

Pooling mode is read from nutribot_adapter_config.json inside the adapter directory ("mean" for bge-m3, "lasttoken" for gte-Qwen2). If loading fails for any reason, the function falls back to the base model with a warning — no crash.

Why fallback instead of hard fail? The embedding adapter is an optional improvement, not a core dependency. A failed adapter load should not take down the production server.

10. Database Layer

10.1 Relational Database (SQLAlchemy)

File: database.py

Connection URL read from DATABASE_URL, defaulting to postgresql://postgres:postgres@localhost:5432/nutribot.

Tables:

api_clients — B2B tenants | Column | Type | Notes | |—|—| | id | Integer PK | Used as client_id throughout | | client_name | String UNIQUE | | hashed_api_key | String UNIQUE | werkzeug PBKDF2 of nbk_live_{hex} | | patients | relationship | → Patient rows (cascade delete) |

document_metadata — Tracks uploaded documents per client | Column | Type | Notes | |—|—| | id | Integer PK | | client_id | Integer FK → api_clients.id | CASCADE DELETE | | filename | String | | file_hash | String | SHA-256 hex, for deduplication | | upload_date | DateTime | UTC | | file_size | Integer | Bytes | | chunk_count | Integer | pgvector chunks created | | status | String | pending / completed / failed |

patients — Patient medical records (added April 2026)

See Section 7.1 for full schema.

10.2 Vector Store (pgvector)

PostgreSQL with pgvector extension. LangChain manages two tables:

langchain_pg_collection — Named collections (one per client + shared base)

langchain_pg_embedding — Chunk rows | Column | Type | Notes | |—|—| | uuid | UUID PK | | collection_id | UUID FK | | embedding | vector(1024) | 1024-dim for bge-m3 (was 384 for bge-small) | | document | Text | Raw chunk text | | cmetadata | JSONB | Includes source, title, file_hash |

Dimension change: After switching from bge-small-en-v1.5 (384-dim) to bge-m3 (1024-dim), the entire vector store was rebuilt. All existing embeddings were deleted and regenerated. The two source document directories were re-ingested:

```
BASE_DOCS_DIR=/home/han/documents_to_ingest python build_base_db.py
BASE_DOCS_DIR=/home/han/documents_to_ingest_new python build_base_db.py
```

11. Document Ingestion Pipelines

11.1 Base Knowledge Build (build_base_db.py)

Standalone script that populates the shared base_knowledge pgvector collection.

Configuration: - BASE_DOCS_DIR: Source directory for documents (env var, defaults to data/base_docs/) - FILE_TRACKER_PATH: data/file_tracker.json — stores {filename: mtime}

for incremental runs - MAX_WORKERS: min(cpu_count, 4) — parallel PDF parsing via ProcessPoolExecutor - DB_BATCH_SIZE: 500 chunks per pgvector insertion batch

Multilingual OCR support (added April 2026):

```
elements = partition(filename=filepath, strategy="fast", languages=["eng",  
"msa"])  
if not elements:  
    elements = partition(filename=filepath, strategy="hi_res", languages=["eng",  
"msa"])
```

languages=["eng", "msa"]: msa is the ISO 639-2 code for Bahasa Melayu, used by Tesseract. This was added when a folder of Malay-language nutrition guidelines was ingested. Without it, Tesseract defaulted to English-only OCR and produced garbled text for Malay characters.

NUL byte sanitisation (added April 2026):

```
content = str(chunk).replace("\x00", "").strip()  
if not content:  
    continue
```

Some PDFs from the second document folder contained binary corruption that Unstructured extracted as null bytes (\x00). PostgreSQL rejects string literals containing null bytes, raising ValueError: A string literal cannot contain NUL (0x00) characters. The strip is applied before every insertion.

Chunking parameters: - max_characters=1500: Maximum chunk size. Chosen to fit within the embedding model's effective context window while remaining large enough to contain complete nutritional recommendations. - combine_text_under_n_chars=500: Prevents very short fragments (e.g., standalone table headers) from becoming useless isolated chunks.

11.2 Client Document Ingestion (process_client_docs.py)

Called within POST /upload_documents/. SHA-256 deduplication, same chunking parameters as base knowledge, inserts into client_{id}_knowledge in batches of 100.

11.3 Document Deletion (document_manager.py)

Raw psycopg2 DELETE filtered by cmetadata->>'file_hash' — targeted chunk deletion without loading data into memory.

12. Multi-Tenancy Model

Each B2B client (ApiClient) gets: - **Isolated pgvector collection:** client_{id}_knowledge - **Isolated patient records:** patients rows filtered by client_id - **Shared base knowledge:** All clients share base_knowledge

The client_id propagates from the API key through every layer:

```
website_chat_router → rag.get_rag_response(client_id)  
                    → chain_factory.create_conversational_chain(client_id)  
                    → vector_store.get_retriever(client_id)
```

Patient records are also scoped: GET /patients/ and GET /patients/{id} both enforce `patient.client_id == client.id`, preventing cross-tenant access.

13. Conversation Memory

File: `chain_factory.py`

Module-level in-memory dict:

```
_session_store: dict[str, InMemoryChatMessageHistory] = {}
```

`RunnableWithMessageHistory` injects history as `chat_history` before each invocation and appends the new turn after.

Limitations: Non-persistent (lost on restart), process-local (one Gunicorn worker only). Production systems should replace with a Redis-backed `BaseChatMessageHistory`.

14. Persona & Prompt Engineering

File: `chain_factory.py` — `get_system_template(target_disease, patient_context="")`

The system prompt structure (in order):

1. **Role definition** — AI Nutrition Assistant acting as a professional dietitian
 2. **Patient profile block** (injected when `patient_context` is non-empty):

```
**Current Patient Profile:**  
Name: ... Age: ... Ethnicity: ... BMI: ...  
Conditions: ... Medications: ... Dietary restrictions: ...
```
 3. **Core persona rules** — Professional, empathetic, collaborative (“we”, “let’s”)
 4. **Open-ended questioning rules** — Explicit bad/good question examples, anti-looping rules
 5. **Malaysian multicultural food context** — Per-ethnicity food references (Malay/Chinese/Indian), mamak culture, communal dining
 6. **ADIME framework** — Assessment → Diagnosis → Intervention → Monitoring & Evaluation
 7. **Image display instructions** — Food-to-filename mapping for [IMAGE:] markers
 8. **RAG context** — {context} placeholder filled at invocation with retrieved chunks
- {target_disease} parameterises the focus throughout the prompt.
-

15. Streaming Response Delivery

File: `website_chat_router.py`

`StreamingResponse` with `media_type="text/event-stream"`. The `async` generator `stream_rag_response` calls `rag.get_rag_response()` synchronously, then yields the full answer string character-by-character with a 10ms delay.

Note: This is post-hoc streaming simulation — the LLM generates the full response before streaming begins. True token streaming would require `chain.astream()`.

16. UI Layers

16.1 Patient-Facing App (`patient_app.html`)

Added April 2026. Served at `GET /`.

Self-contained single-page application — no CDN dependencies, no build step. All CSS, JS, and HTML in one file.

Design decisions:

- **No CDN dependencies:** The app must function on clinic intranets with restricted external access. Inline everything.
- **URL param API key (`?key=`):** Clinics can distribute pre-authenticated URLs to patients (e.g., via QR code) without requiring patients to type an API key manually. The login screen shows a “Connected to clinic” badge when a key is present.
- **IC auto-formatting:** The `fmtIC()` JS function formats input as `YYMMDD-SS-XXXX` as the user types, preventing manual format errors.

Three screens:

Screen 1 — Login: - Name input field - “Connected to clinic” badge if `?key=` is in URL - No password required at this step (patients often forget passwords)

Screen 2 — IC Verification: - IC number input with auto-format - Verify / Back buttons - On success, transitions to Screen 3

Screen 3 — Dashboard + Chat: - Dark green sidebar (`--g900:#0d3d24`) with patient profile pills (conditions, medications, dietary restrictions, allergies) - Chat area with streaming SSE response rendering - Mobile: sidebar slides in from left via `.sbtog` button + dark overlay mask - Session ID generated client-side as a random UUID per page load

Colour palette rationale: - `--g900:#0d3d24` / `--g700:#1a6b45`: Deep green evokes health, clinical professionalism, and Malaysian nature imagery (consistent with national branding conventions) - `--warm:#fafaf8`: Off-white reduces eye strain in clinical reading contexts - `--amber:#f59e0b`: High-contrast accent for interactive elements (accessibility)

16.2 Admin Dashboard

`GET /admin/create-api-key` — HTML page for creating B2B API keys and listing clients. Admin password checked via `ADMIN_PASSWORD` env var.

16.3 Client Portal

`GET /portal/` — Multi-page portal for B2B clients to manage documents, with embedded test chat. Session stored in `localStorage` via 64-char hex token.

17. Embedding Fine-Tuning Pipeline

This pipeline adapts the embedding model to the medical nutrition domain by training on (query, relevant_passage) pairs derived from the actual ingested documents.

17.1 Training Data Generation

File: generate_embedding_training_data.py

Pulls all chunks from the pgvector base_knowledge collection and uses an LLM to generate 2 realistic questions per chunk — questions a patient or dietitian might ask whose answer is contained in that chunk.

Output: - ~/data/embedding_train.jsonl — 90% of pairs (anchor query, positive passage) - ~/data/embedding_val.jsonl — 10% of pairs - ~69,000 total pairs from ~35,000 chunks × 2 questions each

Provider options:

Provider	--provider	Cost	Speed	Quality
OpenAI GPT-4-turbo	openai	~\$34 for full dataset	Fast (batched)	Highest
Local Ollama	ollama	Free	~3-5s/chunk	Sufficient

Why Ollama over OpenAI for this task? OpenAI quota was exhausted (\$0 credit). Beyond cost, the quality difference between GPT-4-generated and Ollama-generated training *questions* is marginal — the training signal comes from the passage content, not the question sophistication. The questions only need to be plausible, not medically perfect.

--resume flag: Appends to existing output files and skips already-processed chunks (estimated by counting existing pairs / QUERIES_PER_CHUNK). Allows interrupted runs to continue without restarting.

17.2 LoRA Fine-Tuning (finetune_embeddings.py)

Model profiles:

Profile	Base model	Params	Architecture	VRAM usage	Est. time (RTX 3050)
bge-m3 (default)	BAAI/bge-m3	570M	Encoder (XLM-RoBERTa)	~2.0 GB	~45–60 min
gte	Alibaba-NLP/gte-Qwen2-1.5B-instruct	1.5B	Decoder (Qwen2)	~2.5 GB	~40 hrs

Why bge-m3 is the practical choice: - The gte-Qwen2-1.5B model trained at 12.5s/step (decoder architecture processes all prior positions at each token, inherently slower). At that rate, 3 epochs = ~40 hours on an RTX 3050. - bge-m3 (encoder, bidirectional attention) trains at ~0.25s/step — roughly 50× faster — bringing training to under an hour. - bge-m3 is already in production: the fine-tuned adapter loads directly without requiring a vector store rebuild (same model, different weights). Actually a vector store rebuild is still required since embeddings change, but no model architecture change is needed. - The MTEB gap (bge-m3 54.9 vs gte-Qwen2 65.4) is on generic English benchmarks. For medical Malay/English, bge-m3's multilingual pre-training and fine-tuning on scientific text gives it a stronger domain starting point.

17.3 Hyperparameter Decisions

LoRA configuration:

Parameter	Value	Rationale
r (rank)	16	Higher rank = more capacity to adapt to medical domain. Rank 8 is the community default for NLP tasks; rank 16 is chosen here because the domain shift (general → medical nutrition) is substantial and the 570M base model has sufficient capacity. Memory cost is minimal (~60MB vs ~30MB for rank 8).
lora_alpha	32	Conventionally set to $2 \times r$. Acts as a scaling factor for LoRA updates; $\alpha/r = 2$ means LoRA updates are scaled to have meaningful magnitude without dominating base weights.
lora_dropout	0.05	Light regularisation to prevent overfitting on the ~69K training pairs. Higher dropout (0.1+) is unnecessary — the training data is diverse enough.
target_modules	["query", "value"]	For bge-m3 (XLM-RoBERTa): Q and V projections in self-attention. These control how the model attends to and combines information. Fine-tuning Q+V (not K) is the standard recommendation for encoder fine-tuning — K shapes the key space which is less domain-specific.

Training configuration:

Parameter	Value	Rationale
batch_size	8	Reduced from initial 16 to avoid OOM on 4GB VRAM. The double-load problem (Transformer wrapper loads model again before swap) consumes extra VRAM during setup.
grad_accumulation	4	Effective batch = 32. MultipleNegativesRankingLoss (in-batch negatives) benefits strongly from large effective batches — more negatives per anchor = stronger contrastive signal.
learning_rate	2e-4	Standard for LoRA fine-tuning. Lower than typical full fine-tuning (1e-5 to 5e-5) because LoRA updates are small; too low and the adapter barely learns, too high and it overshoots the base model's geometry.
warmup_ratio	0.1	10% of steps used for LR warmup. Prevents large gradient steps at the start when the adapter weights are randomly initialised.
MAX_SEQ_LEN	256	Reduced from 512. For retrieval fine-tuning, most meaningful query-passage pairs are under 256 tokens. The reduction halves activation memory with negligible quality impact for this use case.
bf16=True	—	BF16 (Brain Float 16) used instead of FP16. FP16 training uses a GradScaler that fails with already-FP16 LoRA parameters (raises ValueError: Attempting to unscale FP16 gradients). BF16 has the same memory footprint but does not require gradient scaling, bypassing this conflict entirely. RTX 3050 (Ampere architecture) supports

Loss function: MultipleNegativesRankingLoss (MNRL) — treats all other passages in the same batch as negatives for each anchor. No explicit negative mining required. Effective batch size of 32 provides 31 negatives per anchor, which is sufficient for domain adaptation. The loss approximates the InfoNCE objective used in CLIP and SimCSE.

Memory fixes applied: 1. `base_hf_model.enable_input_require_grads()` instead of `prepare_model_for_kbit_training()` — the latter casts all parameters to FP32 (requires ~6GB for 1.5B params), causing immediate OOM. 2. `del word_embedding_model.auto_model` + `gc.collect()` + `torch.cuda.empty_cache()` immediately after replacing the Transformer wrapper's internal model — prevents the duplicate model load from occupying VRAM throughout training. 3. `PYTORCH_ALLOC_CONF=expandable_segments:True` — allows the CUDA allocator to reuse fragmented memory blocks, reducing allocation failures from fragmentation.

18. LLM Fine-Tuning Pipeline

The system includes a complete pipeline for fine-tuning a local LLM to serve as a drop-in replacement for GPT-4-turbo.

18.1 Training Data Generation

File: `generate_training_data.py`

Generates synthetic multi-turn ADIME conversations using GPT-4o. Covers 9 health conditions × 12 Malaysian patient personas. Output: `data/train.jsonl` / `data/val.jsonl` in ShareGPT format.

18.2 LoRA Fine-Tuning (Ollama/Unsloth)

File: `colab_finetune.ipynb` / `Modelfile`

Fine-tunes a Gemma-3-4B base model on the synthetic conversations using Unsloth + TRL on Google Colab. Exports as GGUF (Q4_K_M quantisation) for Ollama inference.

Enable with: `USE_OLLAMA=true, OLLAMA_MODEL=nutribot-lora`

19. Evaluation Framework

File: `eval_ragas.py`

RAGAS evaluation over a ground-truth question set:

Metric	Measures
faithfulness	Is the answer grounded in retrieved context? (Hallucination detection)
answer_relevancy	Does the answer address the question?
context_precision	Are retrieved chunks relevant? (Retrieval precision)
context_recall	Do chunks cover the ground truth? (Retrieval recall)

```
python eval_ragas.py --limit 5 # smoke test
python eval_ragas.py --category "Hypertension"
python eval_ragas.py --out results.json
```

20. Testing Strategy

Unit Tests

- **test_db.py** — SQLite in-memory: API client lifecycle, document metadata CRUD
- **test_api.py** — TestClient with app.dependency_overrides[get_db]: health check, auth rejection

Integration Tests

- **test_bot_accuracy.py** — Live API, answer quality evaluation
- **test_bot_culture.py** — Malaysian food scenario handling

conftest.py sets DATABASE_URL=sqlite:///./data/test.db before any imports.

21. Deployment

Development

```
uvicorn app:app --reload --port 8000
```

Production

```
gunicorn -w 1 -k uvicorn.workers.UvicornWorker app:app --bind 0.0.0.0:8000
```

Single worker required (in-memory session store, chat history).

Post-Fine-Tuning Deployment Checklist

After completing embedding fine-tuning:

1. Add to .env: EMBEDDING_ADAPTER_PATH=~/.models/embedding_lora
2. Rebuild vector store (required — embeddings change):

```
docker exec pgvector-nutribot psql -U postgres -d nutribot \
-c "DELETE FROM langchain_pg_collection WHERE name='base_knowledge';"
```

```
BASE_DOCS_DIR=/home/han/documents_to_ingest python build_base_db.py  
BASE_DOCS_DIR=/home/han/documents_to_ingest_new python build_base_db.py
```

3. Restart server
 4. Verify: `python -c "from embeddings import get_embedding_function; ef=get_embedding_function(); print(len(ef.embed_query('test')))"`
 5. Run: `python test_bot_accuracy.py && python test_bot_culture.py`
-

22. Environment Variables Reference

Variable	Default	Required	Description
OPENAI_API_KEY	—	Yes (if not Ollama)	OpenAI API key
OPENAI_MODEL	gpt-4-turbo	No	OpenAI model name
USE_OLLAMA	false	No	Use local Ollama LLM instead
OLLAMA_MODEL	nutribot-lora	No	Ollama model name
OLLAMA_BASE_URL	http://localhost:11434	No	Ollama server URL
EMBEDDING_MODEL	BAAI/bge-m3	No	HuggingFace embedding model
EMBEDDING_ADAPTER_PATH	—	No	Path to LoRA adapter dir; activates fine-tuned embeddings
FINETUNE_BASE_MODEL	Alibaba-NLP/gte-Qwen2-1.5B-instruct	No	Override base model for finetune_embeddings.py
DATABASE_URL	postgresql://postgres:postgres@localhost:5432/nutribot	Yes (prod)	SQLAlchemy relational DB URL
PGVECTOR_URL	Falls back to DATABASE_URL	Yes (prod)	PostgreSQL URL with pgvector
BASE_DOCS_DIR	data/base_docs/	No	Source PDFs for base knowledge build
PERSISTENT_DISK_PATH	./data	No	Root data directory
ADMIN_PASSWORD	—	Yes (prod)	Admin panel password
REDIS_URL	—	No	Redis URL for optional session caching
PORT	8000	No	Server port

23. Data Flow: End-to-End Request Trace

A single chat request from a logged-in patient (patient_id provided):

1. Patient sends:


```
POST /chat/get_response
Headers: X-API-Key: nbk_live_abcl23...
Body: {
  "question": "What can I eat for breakfast?",
  "session_id": "uuid-xyz",
  "patient_id": 1
}
```
2. FastAPI → `get_api_client()`:
 - Hash-checks API key → `ApiClient(id=2, client_name="test1")`
3. `website_chat_router.py`:
 - `patient_id=1` provided, `profile=None`
 - `db.get_patient(session, 1)` → `Patient(Ahmad Fadzilla, T2DM, Halal, ...)`
 - `db.patient_to_profile_dict(patient)` → profile dict
 - Returns `StreamingResponse(stream_rag_response(..., profile=profile))`
4. `rag.get_rag_response(question, client_id=2, session_id, profile)`:
 - profile provided → skip `identify_target_disease()`
 - `target_disease = "Conditions: Type 2 Diabetes, Hypertension. Medications: Metformin..."`
 - `patient_context = "Name: Ahmad Fadzilla\nAge: 52\nEthnicity: Malay\nBMI: 31.2\n..."`
5. `chain_factory.create_conversational_chain(client_id=2, target_disease, patient_context)`:
 - `get_system_template()` builds prompt with patient block injected
 - `get_retriever("2")` → `MergedRetriever`
6. `vector_store.MergedRetriever._get_relevant_documents("What can I eat for breakfast?")`:
 - `embeddings.py`: encodes query → 1024-dim bge-m3 vector (or fine-tuned adapter vector if `EMBEDDING_ADAPTER_PATH` set)
 - PGVector cosine search: `base_knowledge` → top 5 chunks
 - PGVector cosine search: `client_2_knowledge` → top 5 chunks
 - Merge + deduplicate → ~8 Document objects
7. LCEL chain execution:
 - `context` = concatenated chunk texts
 - `chat_history` = prior turns for "uuid-xyz"
 - Prompt = system(with Ahmad's profile) + history + "What can I eat..."
 - `ChatOpenAI(gpt-4-turbo)` → response mentioning Ahmad by name, Halal options, low-GI foods, no high-sugar Teh Tarik, etc.
8. `parse_response_for_image()` → `{"answer": "...", "image_url": "rice_portions.png"}`
9. `stream_rag_response`: yields answer char-by-char, 10ms delay

10. `_session_store["uuid-xyz"]`: new turn appended for next request

24. Module Reference Table

Module	Role	Key Functions/Classes
app.py	FastAPI app, router registration, patient endpoints	startup_event, patient_login, patient_verify, list_patients
website_chat_router.py	Streaming chat endpoint, patient_id resolution	get_chat_response, stream_rag_response
patient_app.html	Production patient-facing SPA (Login → IC Verify → Chat)	—
admin_router.py	Admin HTML panel for API key management	create_api_key, list_clients
client_portal_router.py	B2B client portal for document management + test chat	authenticate, upload_documents
rag.py	RAG orchestration, patient context building	get_rag_response, identify_target_disease
chain_factory.py	LCEL chain, ADIME prompt with patient block, session memory	create_conversational_chain, get_system_template
vector_store.py	Hybrid multi-tenant retriever	MergedRetriever, get_retriever
embeddings.py	Singleton embedding model, LoRA adapter loading	get_embedding_function, _load_lora_embedding
llm.py	LLM factory (OpenAI / Ollama)	get_llm, get_direct_llm_response
database.py	SQLAlchemy ORM models + CRUD	ApiClient, DocumentMetadata, Patient, patient_to_profile_dict
patient_store.py	PatientStore ABC + SUPPLEMENTARY_FIELDS whitelist	PatientStore, SUPPLEMENTARY_FIELDS
local_patient_store.py	LocalPatientStore (dev/staging) — wraps SQLAlchemy	LocalPatientStore
extractor.py	Passively extracts supplementary fields from patient messages via qwen2.5:32b	extract_from_message, call_ollama_extractor, _validate_extraction, _filter_already_filled
dependencies.py	FastAPI dependency injection	get_db, get_api_client